

Annotated Breaking Changes for the paper *More Effective JavaScript* *Breaking Change Detection via Dynamic Object Relation Graph*

We list our annotated breaking changes from the popular JavaScript libraries as follows. For each breaking change, we provide necessary explanation for ease of comprehension.

async

Version 3.0.0

- In queue, priorityQueue, cargo and cargoQueue, the “event”-style methods, like q.drain and q.saturated are now methods that register a callback, rather than properties you assign a callback to. They are now of the form q.drain(callback). If you do not pass a callback a Promise will be returned for the next occurrence of the event, making them await-able, e.g. await q.drain().

For example, consider the following code:

```
const async = require('async');
var q = async.queue(async fn (task) { /* do the task */ }, 10);
// q.drain is a property and the user can write a function to this property
```

After the code change, the user cannot write the drain property, and drain is a function. The user can set the function as follows:

```
q.drain(() => { });
```

- during and doDuring in async have been removed, and instead whilst, doWhilst, until and doUntil now have asynchronous test functions.
- async.memoize no longer memoizes errors. Consider the following code:

```
const async = require('async');
var e = new Error('test error');
var fn = async.memoize(function(arg0, arg1, callback) {
  console.log('arguments:', arg0, arg1);
  callback(e, {});
});

fn('a', 'b', (err, result) => {
  if (err) {
    console.log('There is an error.')
    e = null;
    fn('a', 'd', function (err, result) {
      if (err) {
        console.log('There is an error.')
      }
      console.log(result)
    })
  }
})
```

In the code above, the function to be memoized should have the last parameter to be a callback function. Following Node.js function convention, the callback function often accepts two arguments, i.e., error and result, respectively. By calling `async.memoize`, we can get a wrapped function (`fn`), which can be used as the original function.

Before the breaking change, if the first call to `fn` generates an error, then the error and the error and the result will be still memoized. Hence, 'There is an error' will be printed twice. However, after the breaking change, the error will not be memoized, hence 'There is an error' will only be printed once. The breaking change is related to issues [#1465](#) and [#1466](#).

- `async.applyEach` and `async.applyEachSeries` have a simpler interface, to make them more easily type-able. It always returns a function that takes in a single callback argument. If that callback is omitted, a promise is returned, making it awaitable.
- Limits of less than 1 (for functions `async.eachLimit`, `concatLimit`, etc.) now cause an error to be thrown in queues and collection methods. For example, the following code will cause an error.

```
const async = require('async');
async.eachLimit([1, 2, 3, 4], 0, function(val, next) { });
```

- Most `async` methods return a Promise when the final callback is omitted, making them awaitable (related to issue [#1572](#)).

For example:

```
const async = require('async');
const fs = require('fs');
const fileList = ['a.txt', 'b.txt'];

function fileExists(file, callback) {
  fs.access(file, fs.constants.F_OK, (err) => {
    callback(null, !err);
  });
}

async.every(fileList, fileExists, function(err, result) {
  // process err and result
});
```

Before the code change, if the third argument (a callback function) when calling `async.every` is not provided, the errors and results will be ignored. After the change, if the third argument is not provided, `async.every` will return a Promise object, and the results can be retrieved by calling `.then()`.

Version 2.0.0

- Calling a callback more than once is considered an error, and an error will be thrown. This had an explicit breaking change in waterfall. For example:

```
const async = require('async');
async.waterfall([
  function(callback) {
    callback(null, 'one', 'two');
  },
  function(arg1, arg2, callback) {
    // arg1 now equals 'one' and arg2 now equals 'two'
    callback(null, 'three');
  },
  function(arg1, callback) {
```

```

        // arg1 now equals 'three'
        callback(null, 'done');
    }
], function (err, result) {
    // result now equals to 'done'
});

```

Previously multiple callbacks would call the rest of the waterfall multiple times. After the breaking change, the callback cannot be called more than once. The breaking change is related to issue [#1050](#).

- `async.auto` task functions now always take the callback as the last argument. If a task has dependencies, the results object will be passed as the first argument.
- `map` used to return an object when iterating over an object. `map` now *always* returns an array, like in other libraries.
- `filter`, `reject`, `some`, `every`, `detect` and their families like `{METHOD}Series` and `{METHOD}Limit` now expect an error as the first callback argument, rather than just a simple boolean. Pass `null` as the first argument, or use `fs.access` instead of `fs.exists`.
- `async.retry`'s callback is now optional. Previously, omitting the callback would partially apply the function, meaning it could be passed directly as a task to `series` or `auto`. The partially applied "control-flow" behavior has been separated out into `retry-able`.
- The `q.tasks` array (`q` is returned from `async.queue()`) has been renamed `q._tasks` and is now implemented as a doubly linked list (DLL). Any code that used to interact with this array will need to be updated to either use the provided helpers or support DLLs. The breaking change is related to issue [#1205](#).

joi

Version 16.0.0

- Remove `isJoi` from schema (use `Joi.isSchema()` instead). Before the breaking changes, the objects created by `Joi` functions (e.g., `joi.string`, `joi.array`) have a property named `isJoi`, indicating that the object is a `Joi` instance. For example:

```

const joi = require('joi');
assert(joi.array().isJoi === true);

```

After the breaking change, the objects no longer have `isJoi` property. The users should use `joi.isSchema` function to determine whether an object is a `Joi` instance. The breaking change is related to issue [#1834](#).

- Only execute coerce extensions when `convert` is `true`.
- Apply `array.single()` regardless of `convert`.
- Change object and array errors to use full path instead of nested error messages.
- Remove `Hoek.reach()` options from `Joi.ref()`.
- Limit `array.single()` to only allow non-array items. The `single` method can be accessed by the objects created by `joi.array()`.
- Forbid chaining `when()` calls when the first was not on alternatives type.
- Remove any `.error()` options. Here `any` refers to any `Joi` types (e.g., `string`, `array`, `number`). Before the breaking change, the user can write the following code:

```
const joi = require('joi');
joi.string().error(new Error('some information'), {});
```

After the breaking change, the second argument (the option object) is ignored. The breaking change is related to issues [#1857](#) and [#1858](#).

- Remove any `createOverrideError()`. Any types of Joi do not have `createOverrideError` property after the breaking change.
- Renamed `options()` to `preferences()`.
- `any.error()` only supports an `Error` or a function that returns a single `Error` (drop support for a function that returns string or array or errors).
- Errors paths only include the edge label.
- Change `object.rename('a', 'b', {override:true})` to not delete existing key b if a does not exist.
- The reference returned from `Joi.ref()` was changed from a function to object with `ref.resolve()` method.
- Move the `escapeHtml` preference under `errors.escapeHtml`.
- Change the format of references in error messages.
- Error messages require `#` prefix for error local context arguments.
- Remove `isJoi` from `Errors.Report`.
- Move `language.wrapArrays` to `errors.wrapArrays` preference.
- Rename `language` preference to `messages`.

Version 15.0.0

- The options of `Joi.string().email()` is changed. The options `checkDNS`, `errorLevel`, `minDomainAtoms`, etc., are no longer supported. The breaking changes is related to issue [#1776](#).

Version 14.0.0

- `stripUnknown:true` defaults to not strip array items.
- Stricter and safer parsing of numbers.
- Fixed output result of `Joi.string().valid(...).insensitive()`.
- Nested paths on `object.{or,nand,and,xor,with,without}()`.
- Lazy schemas are now less lazy by default.
- `array.{min,max,length}` will be overridden by last call. For example, consider the following code:

```
const joi = require('joi');
var schema = joi.array().max(4).max(3);
schema.validate([1, 2, 3, 4]);
```

Before the breaking change, the schema requires the input array have at least 4 elements. However, after the breaking change, at most 3 elements are allowed. Hence the code above will trigger an error.

- Correctly apply labels to alternatives schemas.
- Error message override is also applied to individual errors.

Version 12.0.0

- Disable HTML escaping by default, leave it as an option.

Version 10.0.0

- Fix boolean strings parsing. This is a minor breaking change.
- In the previous versions, with `convert` set to `true`, Joi supported the use of equivalent values for `true` or `false`, namely `"true"`, `"false"`, `"yes"`, `"no"`, `"on"`, `"off"`, `1`, `0`, `"1"` and `"0"`. This is not the case anymore, it will only support `true`, `false`, `"true"` and `"false"` by default, with the possibility to disable string conversion by using `strict()`.

immutable

Version 5.0.0

- Range function needs at least two defined parameters. The following code will not work:

```
var {Range} = require('immutable');
var range = Range(3);
```

Version 4.0.0

- No longer use value-equality within `merge()` and no longer deeply coerce argument to `merge()`. This change is related to issues [#1391](#) and [#1339](#).

Consider the following code:

```
const {Map} = require('immutable');
const date1 = new Date(1234567890000);
const date2 = new Date(1234567890000);
var m = new Map();
m.set('a', date1);
assert(m.merge({a: date1}) === m);
assert(m.merge({a: date2}) === m);
```

Before the breaking change, the two assertions are both OK, while after the breaking change, the second assertion will fail.

- Replace incompatible collections when merging nested data.
- Concat Lists when merging deeply rather overwrite. Consider the following code:

```
const {Map, Set, List} = require('immutable');
const a = Map({a: Map({x: 10, y: 20}), b: List([1, 2, 3]), c: Set([1, 2, 3])});
const b = Map({a: Map({y: 50, z: 100}), b: List([4, 5, 6]), c: Set([4, 5, 6])});
var m = a.mergeDeep(b);
// m.get('b') will be a List containing 4, 5, 6.
```

After the breaking change, `m.get('b')` will be a List containing 1, 2, 3, 4, 5, 6. This breaking change is related to issue [#1840](#).

- Remove `IteratorSequence`, do not attempt to detect iterators in `Seq()`.
- Remove `Seq.of()`. The code below cannot be used after the breaking change:

```
var {Set} = require('immutable');
var set = Set.of('a', 'b');
```

- `KeyedCollection.toArray()` returns array of tuples. Consider the following code:

```
var {Seq} = require('immutable');
var seq = Seq({a: 'A', b: 'B', c: 'C'});
var array = seq.toArray(); // array is ['a', 'b', 'c']
```

After the breaking change, array is `[['a', 'A'], ['b', 'B'], ['c', 'C']]`. This breaking changes is related to issue [#1340](#).

- `list.concat()` now has a slightly more efficient implementation and `map.concat()` is an alias for `map.merge()`. This means that `List.prototype.concat === List.prototype.merge`.
- `Iterable` has been renamed to `Collection`, and `isIterable()` has been renamed to `isCollection()`. Aliases with the existing names exist to make transitioning code easier.
- The sequence API (such as `map`, `filter`, `forEach`) no longer exist on `Records`, `delete()` and `clear()` no longer exist on `Records`. For example:

```
const {Record} = require('immutable');
var R = Record({a: 1, b: 2});
R().map(x => { /* ... */ }); // map function
```

After the breaking change, the object returned by `R()` cannot access `map`.

- Plain Objects and Arrays are no longer considered opaque values.
 - The `toJSON()` method performs a shallow conversion (previously it was an alias for `toJS()`, which remains a deep conversion).
 - Map constructor no longer takes an array, resulting in numeric keys. For example, the code below will trigger an error.
- ```
const {Map} = require('immutable');
var m = Map(['A', 'B', 'C']);
```
- The “predicate” functions, `isCollection`, `isKeyed`, `isIndexed`, `isAssociative` have been moved from `Iterable` to the top level exports. Hence, the user cannot access `Iterable.isCollection`, `Iterable.isKeyed`, etc.

## *uuid*

### **Version 3.0.0**

- Remove `.parse` and `.unparse`. The methods `parse` and `unparse` of the `import` library cannot be accessed any more.

## **execa**

### **Version 9.0.0**

- When the encoding option is “buffer”, the output (result.stdout, result.stderr, result.all) is now an Uint8Array instead of a Buffer.

Consider the following code:

```
var execa = require('execa');
var result = execa('node', ['file.js'], {encoding: 'buffer'});
let {stdout, stderr} = result;
```

After the breaking change, stdout and stderr refer to Uint8Array<sup>1</sup> objects instead of Buffer<sup>2</sup> objects. This breaking change is related to issue #586.

- Renamed some of the allowed values for the encoding option of execa.
- Passing a file path to subprocess.pipeStdout(), .pipeStderr() and .pipeAll() has been removed. Instead, a {file: './path'} object should be passed to the stdout or stderr option.

For example, before the breaking change, the user can use the following code:

```
var execa = require('execa');
var subprocess = execa('node', ['file.js']);
subprocess.pipeStdout('/path/to/a/file');
```

After the breaking change, the user should set the file option when call execa, like this:

```
var execa = require('execa');
var subprocess = execa('node', ['file.js'], {file: '/path/to/a/file'});
```

- Renamed error.killed to error.isTerminated. For example, consider the following code:

```
var execa = require('execa');
try {
 await execa('node', ['file.js']);
} catch (error) {
 // error.killed should be true
}
```

After the breaking change, error should have isTerminated property (value is true) and no killed property.

- Renamed the signal in option to cancelSignal.
- The subprocess.cancel() has been removed. Use either subprocess.kill() or the cancelSignal option instead. Before the breaking change, the user can write the following code:

```
var execa = require('execa');
var subprocess = execa('node', ['file.js']);
subprocess.cancel();
```

After the breaking change, the user should use subprocess.kill(), or set cancelSignal option when call execa.

- Renamed the forceKillAfterTimeout option to forceKillAfterDelay. Also, it is now passed to execa() instead of subprocess.kill(). Consider the code below:

---

<sup>1</sup>See [https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global\\_Objects/Uint8Array](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Uint8Array)

<sup>2</sup>See <https://nodejs.org/api/buffer.html>

```
var execa = require('execa');
execa('node', ['file.js'], {forceKillAfterTimeout: 1000});
```

After the breaking change, the process will not be forced killed after 1 second. Instead, the user should set `forceKillAfterDelay`. The breaking changes is related to issues [#723](#) and [#714](#).

- The `verbose` option is now a string enum instead of a boolean. `false` has been renamed to `none` and `true` has been renamed to `short`.

Consider the following code:

```
var execa = require('execa');
execa('node', ['file.js'], {verbose: false});
```

The value `false` is not accepted by the option `verbose`. Instead, it should be set to `'none'` to retain the previous behavior. The breaking change is related to issue [#884](#).

- The `execPath` option has been renamed to `nodePath`. It is now a noop unless the `node` option is `true`. Also, it now works even if the `preferLocal` option is `false`.
- The default value for the `serialization` option is now `advanced` instead of `json`. In particular, when calling `subprocess.send(object)` with an object that contains functions or symbols, those were previously silently removed. Now this will throw an exception.
- Signals passed to `subprocess.kill()` and to the `killSignal` option cannot be lowercase anymore. Consider the following code:

```
var execa = require('execa');
execa('node', ['file.js'], {killSignal: 'sigterm'});
```

The option value for `killSignal` should be `'SIGTERM'` instead of `'sigterm'`. Hence, when the process receive a actual `SIGTERM` signal (typically when the user press `Ctrl+C` in the keyboard),<sup>3</sup> the spawned process will not be killed.

### Version 3.0.0

- The `all` property is now undefined unless the `all` option is set to `true`.
- `error.exitCodeName` has been removed.

## *find-up*

### Version 4.0.0

- Return undefined instead of null when the path cannot be found.
- Add ability to specify if looking for file or directory, It now only matches files by default, not also directories. Previously it would match either.

## *fast-deep-equal*

### Version 2.0.0

- NaN are now equal. After the breaking change, two NaN values should be equal to each other.
- Different functions are not equal. For example:

---

<sup>3</sup>See [https://en.wikipedia.org/wiki/Signal\\_\(IPC\)](https://en.wikipedia.org/wiki/Signal_(IPC))

```
const equal = require('fast-deep-equal');
var isEqual = equal(function() { }, function() { });
```

After the breaking change, `isEqual` will be false.

## *express*

### Version 4.0.0

- Remove `express.connect` and `connect`'s patches except for charset handling.
- Remove `bodyParser` middleware.
- Remove all bundled middleware except `static`.
- Remove `express.createServer()`. It has been deprecated for a long time. Use `express()`.
- Remove `app.configure` (`app` is returned by calling `express()`). The users should use logic in your their own app code.
- Remove `app.router`.
- Remove `req.auth`. Use `basic-auth` instead.
- Change `req.params` to an object. For example:

```
const express = require('express');
var app = express();
app.get('/', (req, res) => {
 // req.params is an object
})
```

- Remove `req.accepted*`. Use `req.accepts*()` instead.

For example:

```
const express = require('express');
var app = express();
app.get('/', (req, res) => {
 // properties req.accepted* cannot be accessed after the breaking change
});
```

- Remove `res.charset`. Include the charset in the content type when using `res.set()`.
- change `app.route` to `app.mountpath` when mounting an express app in another express app.
- Change `req.accepts*` to `req.accepts*s`, i.e. `req.acceptsEncoding` to `req.acceptsEncodings`.
- `res.locals` is no longer a function. It is a plain JavaScript object. Treat it as such.

## *js-tokens*

### Version 4.0.0

- `'/a/s'.match(jsTokens)` no longer returns `['/', 'a', '/', 's']`, but `['/a/s']`. There are of course other variations of this. Here `jsTokens` is an object created by loading `js-tokens` and can be used as a regular expression object<sup>4</sup>. For example:

---

<sup>4</sup>See [https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global\\_Objects/String/match](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/match)

```
const jsToken = require('js-token');
var result = '/a/s'.match(jsToken);
```

- Code that rely on some token objects not having the closed property could now behave differently.

### **Version 2.0.0**

- `'**'.match(jsTokens)` no longer returns `['*', '*']`, but `['**']`.
- `'**=' .match(jsTokens)` no longer returns `['*', '*=']`, but `['**=']`.

## ***node-fetch***

### **Version 2.0.0**

- `response.text()` no longer attempts to detect encoding, instead always opting for UTF-8 (per spec).
- Make `response.json()` throw error instead of returning an empty object on 204 no-content response.
- Internal methods are no longer exposed.
- Throw error when a GET or HEAD Request is constructed with a non-null body.
- Remove `headers.getAll()`, and make `get()` return all headers delimited by commas.
- Make sure header names and values are valid in HTTP.
- Add `response.arrayBuffer()` (also applies to Requests).
- The “timeout” option was removed.
- `Response.statusText` no longer sets a default message derived from the HTTP status code.
- Dropped the `res.textConverted()` function.
- JSON parsing errors from `res.json()` are of type `SyntaxError` instead of `FetchError`.
- Arbitrary URLs are no longer supported.
- `req.body` can no longer be a string.
- Changed default user agent.

## ***moment***

### **Version 3.0.0**

- `relativeTimeThreshold` keys are now changed to `'s', 'm', 'h', 'd'` and `'M'`. Also the actual thresholds for changed.

## ***jsonwebtoken***

### **Version 9.0.0**

- The `verify()` function no longer accepts unsigned tokens by default.
- RSA key size must be 2048 bits or greater.

## *fill-range*

### Version 3.0.0

1. The `fn` argument of `fillRange` is removed.

## *mongoose*

### Version 6.0.0

- Make connections not thenable, add `Connection#asPromise()` to use a connection as a promise.
- Throw an error if the same query object is executed multiple times.
- Make schema paths declared with type: `{name:String}` create a single nested *subdoc*, and remove `typePojoToMixed` because it is now always false.
- Make `setDefaultsonInsert` (in many functions) true by default. Set to false to disable it.
- Remove context option for queries, always use context query.
- `Aggregate#cursor()` now returns aggregation cursor, rather than aggregate instance.
- Remove `useFindAndModify` option, always use MongoDB's native `findOneAndReplace()` rather than legacy `findAndModify()`.
- Rename `Embedded/SingleNestedPath` -> `Subdocument/SubdocumentPath`.
- call setters with `priorVal` as second parameter, and with new subdocument as context if creating new document.
- Pass document as first parameter to default functions.
- Rename `Query#map()` -> `Query#transform()` to avoid conflating with `Array#map()` and chaining. Hence there is no `mongoose.Query.prototype.map`.
- Remove `Aggregate#addCursorFlag()`, use `Aggregate#options()` instead.
- Make `Aggregate#model()` always return a model rather than an aggregate instance.
- Remove `omitUndefined` option for updates. Mongoose now always removes undefined keys in updates.
- Remove `storeSubdocumentValidationError` schema option, never store subdocument `ValidationErrors`.
- Make `find()` with `explain()` of `mongoose.Query` objects return object instead of an array.
- Allow undefined in number arrays unless required.
- Remove `safe-buffer`, make `MongooseBuffer` extend from native `Buffer`.
- Remove `useNestedStrict`, use nested schema strict mode when casting updates unless `strict` is explicitly specified in query options.
- Make pluralize convert `goose` to `geese`.
- The `Schema#noId` option is now removed, please use `_id` instead. This means that all object (whose prototype is `mongoose.Schema`) will not have `noId` property.
- `createdAt` schema path is now immutable by default.

- Remove `isAsync` option for validators in favor of checking if a function is an async function.
- Make `Document#populate()` return a promise if a callback isn't passed, remove `execPopulate()`.

### Version 5.0.0

- `deleteX()` and `remove()` promise resolves to the write object result.
- `toObject()` and `toJSON()` option parameter merges with defaults rather than overwriting.
- Don't execute getters in reverse order.
- Remove `saveErrorIfNotFound`, always error out if `save()` did not update a document.
- Remove support for `$pushAll`, remove `usePushEach` option.
- Always use mongoose aggregation cursor when using `.aggregate().cursor()`.
- Attach query middleware to prototype rather than in constructor.
- Remove `passRawResult`, only allow `rawResult`.
- `mapReduce` resolves to an object with 2 keys rather than 2 separate args.
- Remove legacy async validator support, and support custom error messages via `throw new Error()`.
- Make `mongoose.connect()` always return a promise.
- Remove `open()`, `openSet()`, `stream()`.
- use `$push` with `$each` instead of `$pushAll`, drop support for `$pushAll`.
- Make document `set()` set keys in the order they're defined in the schema, not in the user specified object.

## **winston**

### Version 3.0.0

- `winston.Logger` has been replaced with `winston.createLogger`.
- `winston.setLevels` has been removed. Levels are frozen at the time of `Logger` creation.
- `winston.transports.Memory` was removed. Use any Node.js `stream.Writable` with a large `highWaterMark` instance instead.
- When writing transports use `winston-transport` instead of `winston.Transport`.
- In `winston.transports.Console`, output for all log levels are now sent to `stdout` by default: `stderrLevels` option now defaults to `[]`, `debugStdout` option has been removed.
- `winston.Container` instances no longer have default `Console` transport.
- `winston.Container.prototype.add` no longer does crazy options parsing.
- `winston.Logger.log` and level-specific methods (`.info`, `.error`, etc.) no longer accepts a callback. The vast majority of use cases for this feature was folks awaiting all logging to complete, not just a single logging message.

- `winston.Logger.add` no longer accepts prototypes or classes. Pass an instance of our transport instead.
- `Logger.prototype.stream: options.transport` is removed. Use the transport instance on the logger directly.
- `Logger.prototype.query: options.transport` is removed. Use the transport instance on the logger directly.
- `winston.exception` has been removed.
- `winston.hash` was removed.
- `winston.common.serialized` was removed.
- `winston.common.log` was removed.
- Removed `winston.transports.{File,Console,Http}` formatting options.
- Migrating filters and rewriters to formats in `winston@3`.

### **Version 2.0.0**

- Move `winston.transports.DailyRotateFile` into a separate module.
- Change the method signature to a filter function to be consistent with rewriter and log functions.